# Dealcore Developer Onboarding

A step-by-step walkthrough for Dax: pull, run, push, deploy, and read production logs.

Repo: github.com/gaslaksen/fast-homes-crm | API on Railway | Web on Vercel

> **What this covers**
> This guide gets you from zero to shipping. Do the one-time setup once (Steps 0 to 2). After that, your everyday loop is Steps 3 to 8, and you can use Claude Code (Step 4) to make the changes. Read CONTRIBUTING.md in the repo for the same rules in text form.

## 0 Get access (one time, ask Geoff first)

Before anything works you need to be invited to three places. Send Geoff your GitHub username and the email you want to use for Railway and Vercel.

- **GitHub** collaborator with **Write** access on `gaslaksen/fast-homes-crm`.
- **Railway** project member (needed to read API logs and env vars).
- **Vercel** project member (needed to read web build and runtime logs).
- The **.env values** shared through a password manager. Never accept secrets over email or chat, and never commit them.

## 1 Install the tools (one time)

You need Node 18 or newer, pnpm 8, Git, and Docker (for a local Postgres and Redis). Install pnpm at the exact version the repo pins:

```
# check versions
node -v      # should be 18 or newer
git --version

# install pnpm 8 (the repo pins pnpm@8.15.0)
npm install -g pnpm@8.15.0
```

## 2 Clone and set up the project (one time)

Clone the repo, install dependencies, create your env files, and set up the local database.

```
# 1. clone and enter the repo
git clone https://github.com/gaslaksen/fast-homes-crm.git
cd fast-homes-crm

# 2. install all workspace dependencies
pnpm install

# 3. start local Postgres + Redis in Docker
docker-compose up -d

# 4. create the API env file, then fill in the secrets
cp apps/api/.env.example apps/api/.env
```

Open `apps/api/.env` and set at minimum `DATABASE_URL` (the docker default works), `JWT_SECRET` (any string locally), and `ANTHROPIC_API_KEY` (powers all AI features). Then create the web env file:

```
# 5. web env: point the frontend at your local API
printf 'NEXT_PUBLIC_API_URL=http://localhost:3001\n' > apps/web/.env.local

# 6. run migrations and seed starter data
pnpm db:migrate
pnpm db:seed
```

> **Keep test mode on locally**
> Set `SMS_TEST_MODE="true"` and `SMS_ALLOWED_NUMBERS` in `apps/api/.env` so local testing never texts a real lead. Twilio is the live SMS provider in production.

## 3 Pull the latest code and run locally

Start every working session by syncing `master`, then start both apps. Do this each day, not just once.

```
# get the newest code before you start working
git checkout master
git pull --rebase origin master
pnpm install        # in case dependencies changed

# start the API (:3001) and web (:3000) together
pnpm dev
```

Then open the app in your browser:

- Frontend: `http://localhost:3000`

- API: `http://localhost:3001`

> **If a schema migration landed while you were away**
> If someone changed the database schema, run `pnpm db:migrate` after pulling so your local database matches the code.

## 4 Make changes with Claude Code (CLI or GUI)

You can use Claude Code, the AI coding agent, to explore this repo and make changes. It works the same from the terminal (CLI) or from an editor, desktop, or web interface (GUI). Either way it reads the code, proposes edits, and asks before it changes files or runs commands. You review every diff and open the pull request yourself. Claude does not push to `master` or deploy for you.

### CLI (terminal)

```
npm install -g @anthropic-ai/claude-code   # one time
cd fast-homes-crm
claude                                      # run from the repo root
```

The first run signs you in with your Anthropic account or Claude subscription. After that, describe what you want in plain English, for example "add a phone field to the lead form and wire it through the API". Claude shows proposed edits and waits for your approval. It can also run the app and tests. The slash command `/code-review` reviews your working changes before you open a PR.

### GUI (editor, desktop, or web)

Same agent, with visual diff review instead of the terminal. Use whichever you prefer:

- VS Code or JetBrains extension (Claude Code inside your editor)
- The Claude desktop app (Mac or Windows)
- The web app at `claude.ai/code`

> **You own the result, not Claude**
> Read every diff before you keep it. Run `pnpm build`, `pnpm test`, and `pnpm lint`. If the schema changed, confirm a migration file was generated and committed. Then branch, commit, and open a PR as in the next step. The repo's `CLAUDE.md` gives Claude the house rules (migrations, no dashes) automatically.

## 5 Make a change and push code

Never work directly on `master`. Create a branch, commit, push it, and open a pull request. This is the only way code reaches production.

```
# 1. branch off an up-to-date master
git checkout master
git pull --rebase origin master
git checkout -b feature/short-description

# 2. ...make your edits, then stage and commit
git add -A
git commit -m "feat: short description of the change"

# 3. push your branch to GitHub
git push -u origin feature/short-description
```

Then open a pull request on GitHub targeting `master`, describe what changed and why, and request a review from Geoff. Once it is approved, merge it.

> **Schema changes REQUIRE a committed migration file**
>
> If you edited `apps/api/prisma/schema.prisma`, run `pnpm db:migrate` and commit the generated migration file with your change. Railway runs `prisma migrate deploy`, which applies migration files but does NOT diff the schema. Forget the file and production silently drifts out of sync and lead queries start crashing.

## 6 Deploy to production

There is no separate deploy button. **Merging your pull request into `master` IS the deploy.** The moment it merges:

- **Railway** rebuilds and redeploys the API (it also runs `prisma migrate deploy` on start).
- **Vercel** rebuilds and redeploys the web frontend.

After merging, watch both dashboards until the new deploy goes green, then click through the live app to confirm your change is there. If something is broken, roll back from the dashboard (Steps 6 and 7 show where) rather than rushing a fix.

## 7 Check Railway logs (the API)

Railway hosts the NestJS API, Postgres, and Redis. Use it to watch a deploy, read runtime errors, or roll back.

### Dashboard route

- Go to `railway.app` and open the Dealcore project.
- Click the **API service**.

- Open the **Deployments** tab to see build and deploy status. Click the active deployment to stream its logs.
- Use the **View Logs** panel for live runtime logs (build logs and deploy logs are separate tabs there).
- To undo a bad deploy: open the last good deployment and choose **Redeploy** / **Rollback**.

### Command line (optional)

```
npm install -g @railway/cli
railway login
railway link          # pick the Dealcore project + API service once
railway logs          # stream live API logs
```

## 8 Check Vercel logs (the web frontend)

Vercel hosts the Next.js frontend. Use it to confirm the web build succeeded and to read runtime errors from the deployed site.

### Dashboard route

- Go to `vercel.com` and open the Dealcore web project.
- Open the **Deployments** tab. Each merge to `master` creates a Production deployment.
- Click a deployment to see **Build Logs** (why a build failed) and **Runtime Logs** (errors from the live site).
- To undo a bad deploy: find the last good deployment, open its menu, and choose **Promote to Production** (rollback).

### Command line (optional)

```
npm install -g vercel
vercel login
vercel logs    # runtime logs for a deployment
```

## Everyday cheat sheet

Once you are set up, this is the whole loop:

```
git checkout master && git pull --rebase origin master
git checkout -b feature/my-change
pnpm dev                      # work locally at localhost:3000
claude                        # optional: make the change with Claude Code
git add -A && git commit -m "feat: my change"
git push -u origin feature/my-change
# open PR -> review -> merge to master -> auto-deploys
# then watch Railway (API) + Vercel (web) go green
```

## The two rules that break production

- **Schema change = migration file.** Run `pnpm db:migrate` and commit the file, or production drifts and lead queries crash.
- **No dashes, anywhere.** House style: never use em dashes or en dashes in code, comments, commits, PRs, or docs. Use a plain hyphen or split the sentence.

---

Questions? Ask Geoff before changing anything that touches live leads, messaging, or the database. Safe default: research, propose, confirm, then change.